1. What is Inheritance?

Inheritance is a mechanism where **one class acquires the properties and behaviors of another class**.

In simple words:

A **child class (derived class)** can reuse the **fields, properties, and methods of a parent class (base class)**.

Real-world example

Think about a **Parent → Child relationship**

- Parent: common features
- Child: inherits parent features + adds its own features

Example:

Vehicle (Parent)

- Start()
- Stop()

Car (Child)

- Start()
- Stop()
- OpenSunroof()

The **Car automatically gets Start() and Stop() from Vehicle**.

---

2. Why Do We Use Inheritance?

Inheritance helps to achieve:

1️⃣ **Code Reusability**

Write code once in the base class and reuse it in multiple classes.

2️⃣ **Maintainability**

Changes in the base class automatically apply to derived classes.

3️⃣ **Extensibility**

Child classes can extend functionality.

4️⃣ **Supports Polymorphism**

Inheritance enables method overriding.

---

3. Syntax of Inheritance in C#

In C#, inheritance is implemented using **colon :**

```
class ParentClass
{
   // members
}

class ChildClass : ParentClass
{
   // additional members
}
```

Here:

- ParentClass → Base class
- ChildClass → Derived class

---

4. Basic Example of Inheritance

Parent Class

```
using System;

class Animal
```

```
{
    public void Eat()
    {
        Console.WriteLine("Animal is Eating");
    }
}
```

Child Class

```
class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog is Barking");
    }
}
```

Main Program

```
class Program
{
    static void Main()
    {
        Dog d = new Dog();


        d.Eat();   // inherited from Animal
        d.Bark();  // defined in Dog
    }
}
```

Output

Animal is Eating

Dog is Barking

So here:

Class	Method
Animal	Eat()
Dog	Bark() + Eat()

The **Dog class inherited Eat() from Animal**.

---

5. Types of Inheritance in C#

C# supports the following types:

Type	Description
Single Inheritance	One parent → One child
Multilevel Inheritance	Grandparent → Parent → Child
Hierarchical Inheritance	One parent → Multiple children

⚠️ **C# does NOT support Multiple Inheritance with classes** (to avoid ambiguity).
But it can be achieved using **Interfaces**.

---

6. Single Inheritance

One base class → One derived class.

Example

```
class Person
{
    public void DisplayName()
    {
        Console.WriteLine("Person Name");
```

```
    }
}

class Student : Person
{
    public void ShowRollNo()
    {
        Console.WriteLine("Roll No: 101");
    }
}
```

Usage:

```
Student s = new Student();
s.DisplayName();
s.ShowRollNo();
```

---

7. Multilevel Inheritance

A class inherits from another derived class.

Person

↓

Employee

↓

Manager

Example

```
class Person
{
    public void ShowPerson()
```

```
    {
        Console.WriteLine("This is Person");
    }
}

class Employee : Person
{
    public void ShowEmployee()
    {
        Console.WriteLine("This is Employee");
    }
}

class Manager : Employee
{
    public void ShowManager()
    {
        Console.WriteLine("This is Manager");
    }
}
```

Usage:

```
Manager m = new Manager();

m.ShowPerson();
m.ShowEmployee();
m.ShowManager();
```

---

8. Hierarchical Inheritance

One parent → Multiple child classes.

```
      Person
     /     \
  Student   Teacher
```

Example

```
class Person
{
    public void Display()
    {
        Console.WriteLine("I am a Person");
    }
}

class Student : Person
{
    public void Study()
    {
        Console.WriteLine("Student is studying");
    }
}

class Teacher : Person
{
    public void Teach()
```

```
    {
        Console.WriteLine("Teacher is teaching");
    }
}
```

---

9. Access Modifiers and Inheritance

Not all members are accessible in derived classes.

Modifier	Accessible in Child Class
private	❌ No
protected	✅ Yes
public	✅ Yes
internal	Depends on assembly

Example using protected

```
class Parent
{
    protected int age = 40;
}

class Child : Parent
{
    public void ShowAge()
    {
        Console.WriteLine(age);
    }
}
```

---

11. Base Keyword in Inheritance

base is used to access **parent class members**.

Example:

```
class Person
{
    public string name = "John";
}

class Student : Person
{
    public void Show()
    {
        Console.WriteLine(base.name);
    }
}
```

---

12. Constructor and Inheritance

When a child object is created:

1️⃣ Parent constructor executes first
2️⃣ Then child constructor executes

Example:

```
class Parent
{
    public Parent()
```

```
    {
        Console.WriteLine("Parent Constructor");
    }
}

class Child : Parent
{
    public Child()
    {
        Console.WriteLine("Child Constructor");
    }
}
```

Output

Parent Constructor

Child Constructor

---

13. Real-Time Example (Healthcare Domain)

Since you like **domain-based examples**, here’s a healthcare example.

Base Class

```
class HospitalStaff
{
    public void Work()
    {
        Console.WriteLine("Staff working in hospital");
    }
}
```

Derived Classes

```
class Doctor : HospitalStaff
{
    public void TreatPatient()
    {
        Console.WriteLine("Doctor treating patient");
    }
}

class Nurse : HospitalStaff
{
    public void AssistDoctor()
    {
        Console.WriteLine("Nurse assisting doctor");
    }
}
```

Usage

```
Doctor d = new Doctor();
d.Work();
d.TreatPatient();

Nurse n = new Nurse();
n.Work();
n.AssistDoctor();
```